Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

Aluno: Arthur de Lima Von Sohsten

E-mail: alvs@cesar.school

GitHub: https://github.com/arthurvonsohsten/Infraestrutura-de-Software---Implementa-o-5-

1. Objetivo (3–5 linhas)

O enunciado pede as funções de sincronização do embarque no metrô da METROREC. Cada carro e cada passageiro roda numa thread própria, e só podem ser usados mutex e variáveis de condição do pthreads. Entrego o sync_car.c com station_init, station_load_car, station_wait_for_car e station_on_board, a struct station no sync_car.h, um Makefile que gera o executável sync_car e um teste extra para casos limite.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Definir struct station e station_init	OK	sync_car.h e sync_car.c
station_load_car só retorna com todos sentados e (carro cheio ou ninguém esperando)	OK	./sync_car não mostra "returned early" nem "Many passengers"
station_load_car retorna na hora sem passageiros ou com 0 assentos	OK	começo do ./sync_car (alarme de 1 s e 2 s) e make limites
station_wait_for_car só retorna com carro na estação e assento livre	OK	make limites: com carro de 0 ou -1 assentos ninguém embarca
station_on_board avisa o carro	OK	./sync_car termina com Looks good!
Usar só as funções pthread listadas	OK	grep -n pthread_ sync_car.c
Sem semáforos	OK	grep -n sem_ sync_car.c não retorna nada
Um único lock por estação	OK	struct station tem um só pthread_mutex_t
Nunca mais de um carro por vez, mesmo destino	OK	premissa usada no signal do on_board
Embarque simultâneo	OK	o teste do professor só chama on_board depois de vários wait_for_car retornarem
Sem busy-wait	OK	toda espera é pthread_cond_wait dentro de while
Makefile gerando sync_car	OK	make clean && make
README com arquivos, compilação, execução, testes e SO	OK	alvs/README.md

3. Como Reproduzir

Compilar:

```
cd alvs
make clean
make
```

Executar (exemplo):

```
./sync_car
```

Testar:

```
make test      # roda ./sync_car 100 vezes
make limites   # casos limite e inválidos(apenas no repositorio do git hub, nao esta no .tar)
```

O sync_car não recebe argumentos.

4. Arquitetura (resumida)

Arquivos e responsabilidades:

1. **sync_car.h → declara a struct station e os protótipos**
2. **sync_car.c → as quatro funções de sincronização**
3. **sync_car-runner.c → teste do professor, sem alteração**
4. **teste_limites.c → meus casos limite e inválidos (está no GitHub; não vai no .tar, que tem só o teste do professor)**
5. **Makefile → all, test, clean(o alvo limites so na versao do github)**

Decisões técnicas:

1. **struct station no .h → o teste cria a estação na pilha e precisa do tipo completo → o .c inclui o .h, e a struct não fica escrita dentro do .c**
2. **broadcast no load_car → todos os passageiros esperando precisam acordar para embarcar juntos → os que não conseguem assento voltam a dormir no while**
3. **reservar o assento no wait_for_car e contar passageiros_embarcando → permite sair do wait_for_car antes do on_board → o carro precisa esperar esse contador chegar a 0**
4. **zerar assentos_livres quando o carro sai → um passageiro acordado tarde não pode pegar assento num carro que já foi → fora do load_car ninguém consegue embarcar**
5. **count negativo vira 0 → entrada inválida não pode liberar passageiro → o carro sai na hora**

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	mutex + 2 conds + 3 contadores	um contador por estado do passageiro (esperando, embarcando) e um para os assentos livres do carro atual	não houve troca
S2			
S3			

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Declarar a struct station com 1 mutex, 2 variáveis de condição e 3 contadores	OK	—	28/10 09:44	commit b789c1c
2	S1	Implementar as 4 funções (broadcast no	OK	—	28/10 14:51	commit c0f20a3

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
		load_car, signal no on_board)				
3	S1	Adicionar o teste do professor e o Makefile	OK	—	29/10 14:34	commit 3b32164
4	S1	Tratar count negativo e criar teste de casos limite	OK	—	03/11 09:52	commit 33c44d9
5	S1	Compilar do zero (make clean && make)	OK, sem nenhum warning	—	03/11, sessão 09:53–10:02	evidencias.log
6	S1	Rodar ./sync_car	OK, 5 carros e Looks good!	—	03/11, sessão 09:53–10:02	evidencias.log, Figura 2
7	S1	make test (100 execuções)	OK, 100 execucoes OK	—	03/11, sessão 09:53–10:02	evidencias.log
8	S1	make limites (8 casos limite e inválidos)	OK, Todos os casos passaram	—	03/11, sessão 09:53–10:02	evidencias.log
9	S1	Rodar ./sync_car 1000 vezes num laço	OK, nenhum FALHA	—	03/11, sessão 09:53–10:02	evidencias.log
10	S1	Compilar com -fsanitize=thread	Parcial: o sanitizer acusou 3 data races e 2 thread leaks	Todos estão em sync_car-runner.c (linhas 25, 87, 108, 111 e 137), que é o teste do professor. Nenhum está no sync_car.c.	03/11, sessão 09:53–10:02	evidencias.log, Figura 1
11	S1	Conferir com grep os requisitos de primitivas	OK: nenhum sem_, 1 único pthread_mutex_t, só funções pthread permitidas	—	03/11, sessão 09:53–10:02	evidencias.log

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

O arquivo evidencias.log está dentro do alvs.tar e no repositório (alvs/evidencias.log). Gravei com script -a evidencias.log no Ubuntu (WSL2), na sessão de 03/11/2026 das 09:53 às 10:02, usuário arthur, pasta

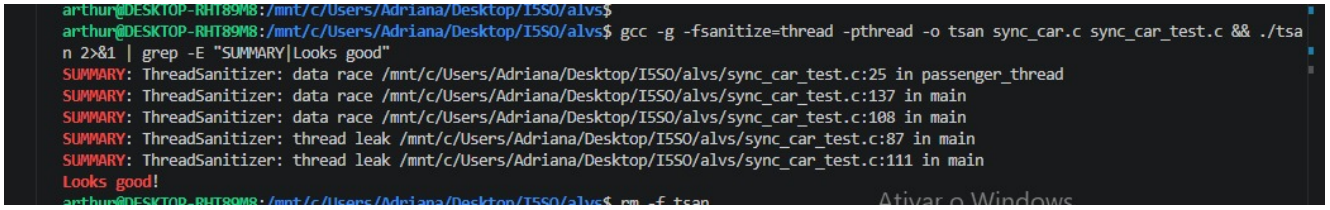
/mnt/c/Users/Adriana/Desktop/I5SO/alvs. Ambiente: Linux 6.18.54.1-microsoft-standard-WSL2 e gcc 15.2.0 (Ubuntu).

6.2 Prints

- Erro principal: o momento do bug mais relevante da semana.
- Validação final: teste passando / saída correta.

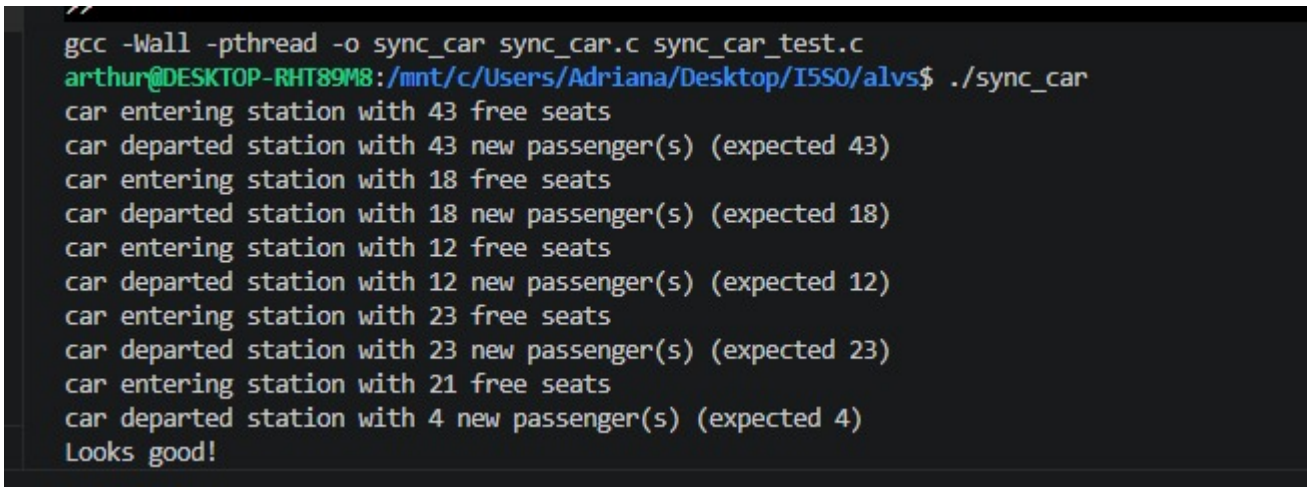
O teste do professor foi renomeado de sync_car_test.c para sync_car-runner.c (nome original) depois da gravação do evidencias.log e da Figura 1. O conteúdo é o mesmo; por isso o log e o print mostram o nome antigo

FIGURA 1



Saída do ThreadSanitizer (gcc -fsanitize=thread). Os 3 data races (linhas 25, 108 e 137) e os 2 thread leaks (linhas 87 e 111) estão todos em sync_car-runner.c, o arquivo de teste fornecido. Os data races vêm das variáveis globais threads_completed e load_car_returned, lidas sem lock. Os thread leaks vêm das threads criadas sem pthread_join. Nenhum aviso aponta para o sync_car.c, e o teste termina com Looks good!.

FIGURA 2



Validação final. Execução de ./sync_car: 5 carros com assentos aleatórios, todos com a quantidade de passageiros esperada, terminando em Looks good!.

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

7. Uso de IA

Onde usei IA (2–3 linhas):
Usei o Claude Code (Anthropic) para escrever a implementação do sync_car.c, o Makefile, o teste de casos limite e os READMEs, para organizar as etapas de commit e para montar o rascunho deste relatório.

Prompts principais (liste, sem história):

- enunciado da atividade e arquivo de teste colados, pedindo uma implementação simples, em português, separada em etapas
- "me guie para criar o repositório da forma correta"
- "aqui localmente, já está pronta a implementação completa de código?"
- guia de relatórios colado, pedindo ajuda para preencher

- "me explique o código com palavras simples, arquivo por arquivo, linha por linha"
- "me guie para fazer o evidências"
- "preencha a parte do relatório que falta com as informações que conseguiu"

O que validei manualmente e como:
Rodei eu mesmo todos os testes no Ubuntu (WSL2), gravando com script (evidencias.log): compilação do zero sem warnings, ./sync_car com Looks good!, make test com 100 execuções, make limites com 8 casos e um laço de 1000 execuções sem falha. Conferi com grep que não há semáforo, que existe só um mutex e que só aparecem as funções pthread permitidas. Rodei o ThreadSanitizer e conferi que os avisos estão só no arquivo de teste do professor.

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão: A parte mais difícil foi entender quando o carro pode sair da estação.Não dava para olhar só os assentos, porque às vezes o carro tem mais lugar do que gente esperando.Também não dava para olhar só a fila, porque um passageiro pode já ter pegado o assento e ainda não ter sentado.Por isso usei três contadores: quem está esperando,quem está embarcando e quantos assentos sobram.Usei broadcast para acordar todos os passageiros de uma vez e signal para avisar o carro, já que só tem um carro por vez.O ThreadSanitizer me assustou no começo, mas olhando as linhas vi que os avisos eram todos do arquivo de teste e não do meu código.Na próxima eu escreveria os testes de casos limite antes da implementação

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim: make clean && make no evidencias.log
Rodei os testes e evidencias.log foi gerado	Sim: sessão de 03/11, 09:53–10:02
Tenho prints obrigatórios	Sim: Figura 1 (erro principal) e Figura 2 (validação final)
Testei pelo menos um caso limite e um caso inválido	Sim: make limites, com 0 assentos, ninguém esperando e lotação exata (limite) e assentos negativos (inválido)
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico.
Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

Faria o make test rodar também o teste_limites, e criaria um make tsan para o ThreadSanitizer.

Rodaria o teste com um núcleo só (taskset -c 0), que não cheguei a fazer.

No teste_limites.c, trocaria o usleep(100000) por uma espera de verdade, porque numa máquina lenta o teste pode falhar sem o código estar errado.

Testaria com 1000 passageiros e compilaria com -Wextra para pegar mais avisos

Testaria no macOS, que o enunciado também cita.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.